

Resumen de clase

Selección de Personal

Single/Multiple Dispatch



2° cuatrimestre 2008

Índice

ENUNCIADO	3
OBJETOS CANDIDATOS.....	4
ENCARANDO UNA SOLUCIÓN... ..	4
<i>LA POSTULACIÓN.....</i>	<i>4</i>
<i>LAS OPCIONES SON... ..</i>	<i>5</i>
<i>Poner un instanceof en cada búsqueda.....</i>	<i>5</i>
<i>Reemplazar los instanceof por métodos isEmpleadoDePlanta(), etc.....</i>	<i>5</i>
<i>Sobrecargar la postulación en cada búsqueda</i>	<i>6</i>
<i>¿Que llame Pekerman?</i>	<i>7</i>
DIAGRAMA DE CLASES	9
ALGO DE CÓDIGO	9
MECANISMOS DE DISPATCH.....	12
<i>SINGLE DISPATCH</i>	<i>12</i>
<i>DOUBLE DISPATCH</i>	<i>12</i>
<i>MULTIPLE DISPATCH.....</i>	<i>13</i>

Enunciado

La Gerencia de Recursos Humanos de “Rabufetti Inc.” quiere informatizar las búsquedas de personal para cubrir sus puestos de trabajo. El informe del relevamiento es el siguiente:

Los postulantes son las personas que acceden al proceso de selección de un cargo, en principio pueden ser:

- **Personal de planta:** son las personas que están en relación de dependencia con la empresa, cobran mensualmente un sueldo fijo determinado por el cargo que ocupan (jefe de planta, supervisor, operario junior, operario semi-senior, gerente línea media, gerente ejecutivo, etc.) y trabajan en un sector (Administración, Sistemas, Recursos Humanos, etc.)
- **Personal contratado que trabaja para la compañía:** son empleados que están en relación de dependencia con otra consultora. Rabufetti paga los servicios de estas personas por hora trabajada pero cada empleado cobra un sueldo propio (que no depende del escalafón de cargos de Rabufetti). Las personas contratadas trabajan en un sector y dependen de un jefe que forma parte del personal de planta de la empresa.

Tanto el personal de planta como los contratados pueden tener empleados a cargo.

- **Externos:** son personas desempleadas o que están trabajando para otra compañía, que dejaron sus datos en una web: nombre y apellido, teléfono, e-mail, su currículum (conformado por las empresas en las que trabajaron + el cargo que ocuparon), fecha de nacimiento, etc.

La empresa realiza distintos tipos de búsqueda:

- **Búsquedas internas:** participan solamente
 - personal de planta
 - personal contratado si la búsqueda es para el mismo sector en el que trabajan.
- **Búsquedas externas:** solamente pueden participar
 - los externos
 - el personal contratado que tenga menos de un año de trabajo en la empresa.
- **Búsquedas especiales:** se utiliza para puestos de alta jerarquía. En ese caso pueden participar:
 - Personal de planta con sueldo actual menor al que se ofrece (según el cargo a elegir) y que tenga al menos 10 personas a cargo
 - Personal contratado que tenga más de 20 personas a cargo
 - Externos que hayan trabajado anteriormente en el mismo puesto

Se pide resolver la postulación de un empleado a una búsqueda

```
public void postular(Postulante postulante)
```

Debe contemplar las validaciones arriba descriptas y agregar la postulación a la lista que mantiene el empleado.

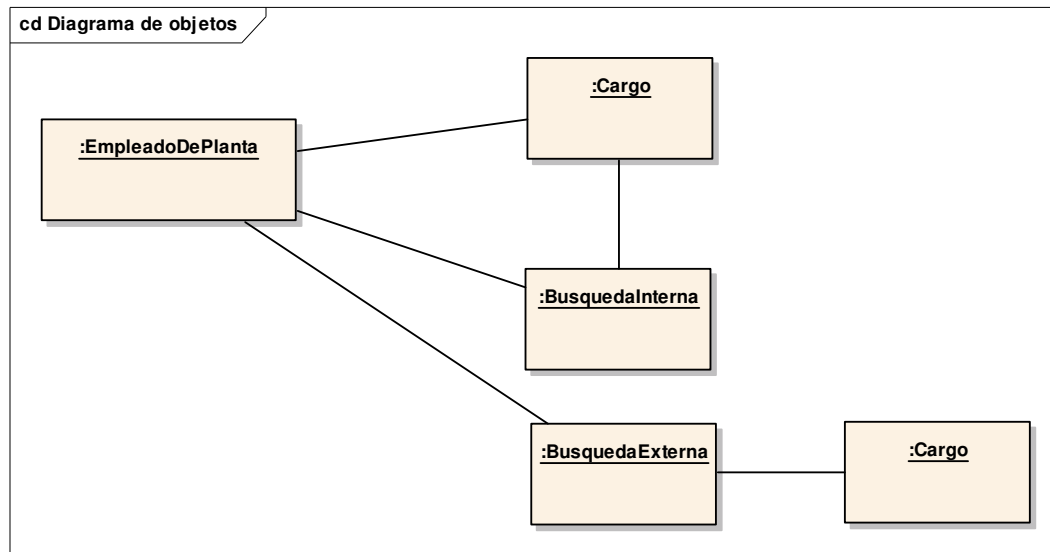
Objetos candidatos

Leemos el enunciado y buscamos los objetos que forman parte del dominio. ¿Qué objetos candidatos aparecen?

- Postulante (personal de planta, contratado, externo)
- Jefe
- Búsqueda (interna, externa, especial)
- Cargo
- Sueldo
- Sector
- Empresa

Encarando una solución...

Hacemos un diagrama de objetos para entender las relaciones:



Decisiones que aparecen:

- Debe o no existir un objeto Cargo → ¿tiene sentido? Sí, el cargo determina el sueldo que cobra.
- Debe o no haber una jerarquía para empleados y búsquedas
- El empleado sabe las búsquedas a las que se postuló, la búsqueda conoce los postulantes o permite navegabilidad en ambos sentidos

La postulación

¿Qué pasa cuando alguien se postula?

- 1) Hay que validar la postulación
- 2) Agregar la postulación al empleado y/o
- 3) Agregar la postulación del empleado a la búsqueda

¿Quién es responsable de cada acción? Pensamos un poco. En 2) es el empleado, y en 3) es la búsqueda.

Con 1) seguramente se va a armar un pequeño debate: la cuestión es que tanto el empleado como la búsqueda son responsables

#Busqueda

```
public void postular(Postulante postulante) {  
    this.validarPostulacion(postulante);  
    postulante.agregarPostulacion(this);  
}
```

Veamos qué opciones se nos ocurren para validar la postulación...

Las opciones son...

Poner un instanteOf en cada búsqueda

#BusquedaInterna

```
public void validarPostulacion(Postulante postulante) {  
    if (postulante instanceof EmpleadoDePlanta) {  
        ...  
    }  
    if (postulante instanceof Externo) {  
        ...  
    }  
    if (postulante instanceof Contratado) {  
        ...  
    }  
}
```

Lo mismo en BusquedaExterna y BusquedaEspecial.

Reemplazar los instanceof por métodos isEmpleadoDePlanta(), etc.
que se redefinen para cada subclase de Postulante.

#BusquedaInterna

```
public void validarPostulacion(Postulante postulante) {  
    if (postulante.isEmpleadoDePlanta()) {  
        ...  
    }  
    if (postulante.isExterno()) {  
        ...  
    }  
    if (postulante.isContratado()) {  
        ...  
    }  
}
```

Esto no nos salva de mucho: la solución parece ser polimórfica, pero cuando las condiciones del negocio cambian, el impacto afecta a la búsqueda. La esencia de delegar y que no me

importe se pierde... lo único que estoy delegando es la pregunta, lo que sigue al if es código (responsabilidad) que sigue quedando en la búsqueda.

Lo único que gané es la posibilidad de decir que un determinado postulante es contratado aunque no pertenezca a la clase Contratado.

Sobrecargar la postulación en cada búsqueda

Tendríamos entonces varias definiciones de postularse:

#BusquedaInterna

```
public void validarPostulacion(EmpleadoDePlanta postulante) {  
    ...  
}  
  
public void validarPostulacion(Externo postulante) {  
    ...  
}  
  
public void validarPostulacion(Contratado postulante) {  
    ...  
}
```

El problema es que las búsquedas dejan de ser polimórficas en la postulación:

```
Postulante alf = new Contratado();  
Busqueda busquedaProgramador = new BusquedaEspecial();  
busquedaProgramador.validarPostulacion(alf);
```

no compila. O sea, no puedo recibir cualquier búsqueda y trabajarla polimórficamente (la sobrecarga trabaja en tiempo de *compilación*, entonces al compilar tengo que saber con qué tipo de postulante voy a trabajar)

De hecho, no nos va a compilar el método postular() de Busqueda porque recibe un postulante:

```
public void postular(Postulante postulante) {  
    this.validarPostulacion(postulante);  
    postulante.agregarPostulacion(this);  
}
```

Y finalmente para que nos compile este código, nos vamos a ver forzados a escribir algo como:

```
public void validarPostulacion(Postulante postulante) {  
    if (postulante instanceof EmpleadoDePlanta) {  
        this.validarPostulacion((EmpleadoDePlanta) postulante);  
    }  
    if (postulante instanceof Contratado) {  
        this.validarPostulacion((Contratado) postulante);  
    }  
    if (postulante instanceof Externo) {  
        this.validarPostulacion((Externo) postulante);  
    }  
}
```

E imaginemos lo difícil que resulta el seguimiento del código para entender qué es lo que hace.

¿Que llame Pekerman?

Ok, no nos gusta ninguna de las opciones anteriores.

Cuando un empleado de planta se postula a una búsqueda interna el objeto búsqueda valida que el postulante cumpla los requisitos. Cada búsqueda sabe si es interna, externa o especial, pero necesita saber qué tipo de postulante es para ver qué validación hacer. Ok, entonces hago:

#BusquedaInterna

```
public void validarPostulacion(EmpleadoDePlanta postulante) {  
    postulante.validarPostulacion(this);  
}
```

Mmm... pero estoy en la misma. Ahora se qué tipo de postulante soy (Contratado, de Planta o Externo), pero me falta saber a qué tipo de búsqueda me estoy postulando.

Entonces decimos: ¿qué onda si en el método validarPostulacion() de BusquedaInterna delegamos la validación al postulante pero dándole el contexto de que se valide la postulación a una búsqueda interna?

#BusquedaInterna

```
public void validarPostulacion(EmpleadoDePlanta postulante) {  
    postulante.validarPostulacionABusquedaInterna(this);  
}
```

Lo mismo, en BusquedaExterna podríamos decirle al postulante que valide una postulación pero a una búsqueda externa:

#BusquedaExterna

```
public void validarPostulacion(EmpleadoDePlanta postulante) {  
    postulante.validarPostulacionABusquedaExterna(this);  
}
```

Y por último en la BusquedaEspecial delegamos al postulante con otro método específico que valida una postulación a una búsqueda especial:

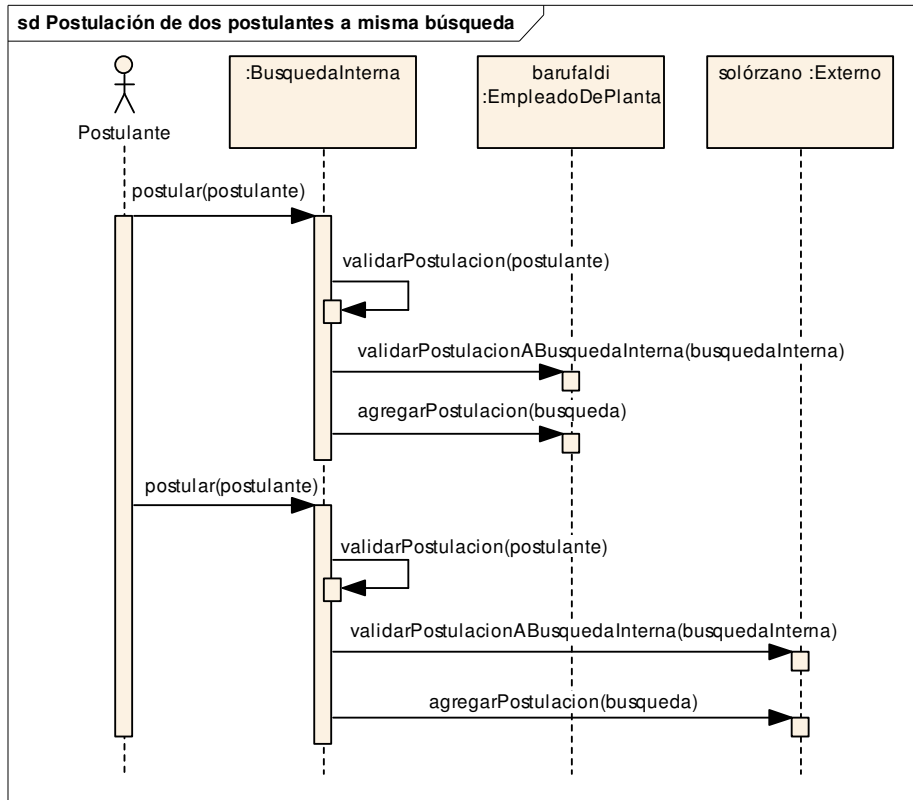
#BusquedaExterna

```
public void validarPostulacion(EmpleadoDePlanta postulante) {  
    postulante.validarPostulacionABusquedaEspecial(this);  
}
```

Lo interesante es que estos 3 métodos:

```
validarPostulacionABusquedaInterna()  
validarPostulacionABusquedaExterna()  
validarPostulacionABusquedaEspecial()
```

son **polimórficos** para los postulantes.



Es decir, cada búsqueda delega al postulante pero agregando información de contexto para ayudar a resolver el requerimiento, porque la responsabilidad de validar una postulación es:

- Del postulante
- De la búsqueda.

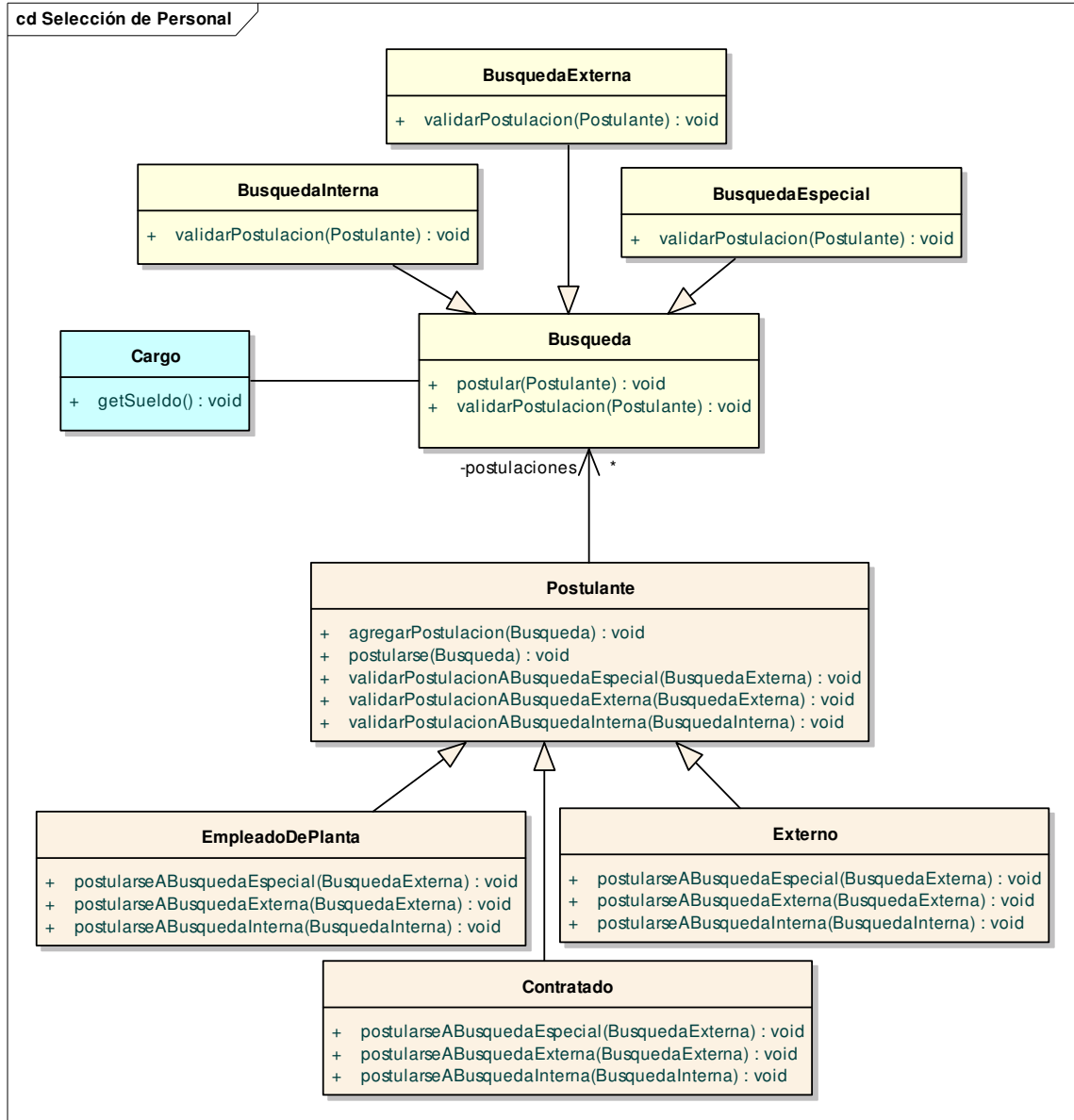
¿La contra? Si agrego otro tipo de búsqueda hay que toquetear en todas las subclases de Empleado.

No me molestaría tanto si agrego un nuevo tipo de postulante...

Si según el negocio es más factible generar nuevos tipos de búsqueda, podría cambiar mi solución para que el postulante delegara la validación a las búsquedas polimórficas mediante métodos:

- validarPostulacionDeEmpleadoDePlanta()
- validarPostulacionDeExterno()
- validarPostulacionDeContratado()

Diagrama de clases



(no todas las clases aparecen, preferí dejarlo menos completo pero más entendible)

Algo de código

(#Busqueda)

```
public void postular(Postulante postulante) {
    validarPostulacion(postulante);
    postulante.agregarPostulacion(this);
}
```

(#BusquedaInterna)

```
public void validarPostulacion(Postulante postulante) {
    postulante.validarPostulacionABusquedaInterna(this);
}
```

```
(#BusquedaExterna)
public void validarPostulacion(Postulante postulante) {
    postulante.validarPostulacionABusquedaExterna(this);
}

(#BusquedaEspecial)
public void validarPostulacion(Postulante postulante) {
    postulante.validarPostulacionABusquedaEspecial(this);
}

(#EmpleadoDePlanta)
public void validarPostulacionABusquedaInterna(BusquedaInterna busqInterna) {
}

(#Postulante)
public void validarPostulacionABusquedaInterna(BusquedaInterna busqInterna) {
    throw new ValidationException("Un " + this.getClass().getName() + " no puede postularse a una búsqueda interna");
}

(#EmpleadoContratado)
public void validarPostulacionABusquedaInterna(BusquedaInterna busqInterna) {
    if (!busqInterna.getSector().equals(sector)) {
        throw new ValidationException("Un empleado contratado no puede postularse a una búsqueda de otro sector");
    }
}

(#Externo)
public void validarPostulacionABusquedaExterna(BusquedaExterna busqExterna) {
}

(#Postulante)
public void validarPostulacionABusquedaExterna(BusquedaExterna busqExterna) {
    throw new ValidationException("Un " + this.getClass().getName() + " no puede postularse a una búsqueda externa");
}

(#EmpleadoContratado)
public void validarPostulacionABusquedaExterna(BusquedaExterna busqExterna) {
    if (getAntigüedad() < 1) {
        throw new ValidationException("El empleado contratado no cumple un año de antigüedad");
    }
}

(#Postulante)
public void validarPostulacionABusquedaEspecial(BusquedaEspecial busquedaEspecial) {
    throw new ValidationException("Un " + this.getClass().getName() + " no puede postularse a una búsqueda especial");
}
```

(#Externo)

```
public void validarPostulacionABusquedaEspecial(BusquedaEspecial busquedaEspecial) {  
    if (!trabajó(busquedaEspecial.getCargo())) {  
        throw new ValidationException("El empleado externo no trabajó en un puesto similar  
anteriormente");  
    }  
}
```

... el desarrollo de trabajó queda para el lector...

(#EmpleadoContratado)

```
public void validarPostulacionABusquedaEspecial(BusquedaEspecial busquedaEspecial) {  
    if (getCantidadPersonasACargo() <= 20) {  
        throw new ValidationException("El empleado externo no tiene más de 20 empleados a  
cargo");  
    }  
}
```

(#EmpleadoDePlanta)

```
public void validarPostulacionABusquedaEspecial(BusquedaEspecial busquedaEspecial) {  
    if (getCantidadPersonasACargo() <= 10) {  
        throw new ValidationException("El empleado externo no tiene más de 10 empleados a  
cargo");  
    }  
    if (!busquedaEspecial.mejora(getSueldo())) {  
        throw new ValidationException("La búsqueda no mejora el sueldo del empleado de  
planta");  
    }  
}
```

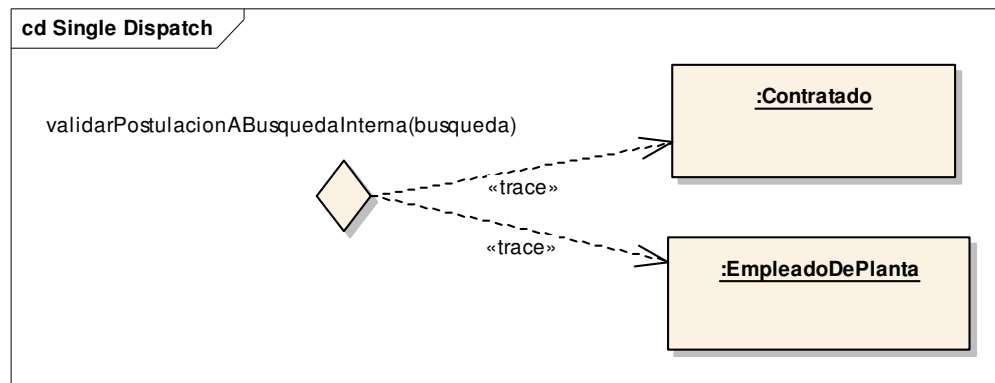
Mecanismos de dispatch

Single Dispatch

Cuando enviamos un mensaje a un objeto, la forma en que sabemos qué método se ejecuta depende del objeto receptor.

```
postulante.validarPostulacionABusquedaInterna(this);
```

Esto se conoce como *single dispatch* → El compilador valida los tipos de los parámetros y el binding dinámico se utiliza para conectar el mensaje con el código a ejecutar en base a la clase concreta del objeto receptor.



Double dispatch

¿Qué pasa cuando dos objetos de distinta jerarquía colaboran para lograr un objetivo común?

Necesitamos implementar un double dispatch: cuando envío el mensaje a un objeto, en realidad estoy enviando dos mensajes al mismo tiempo.

Consideremos el ejemplo que muestra cómo se imprime un objeto en Smalltalk:

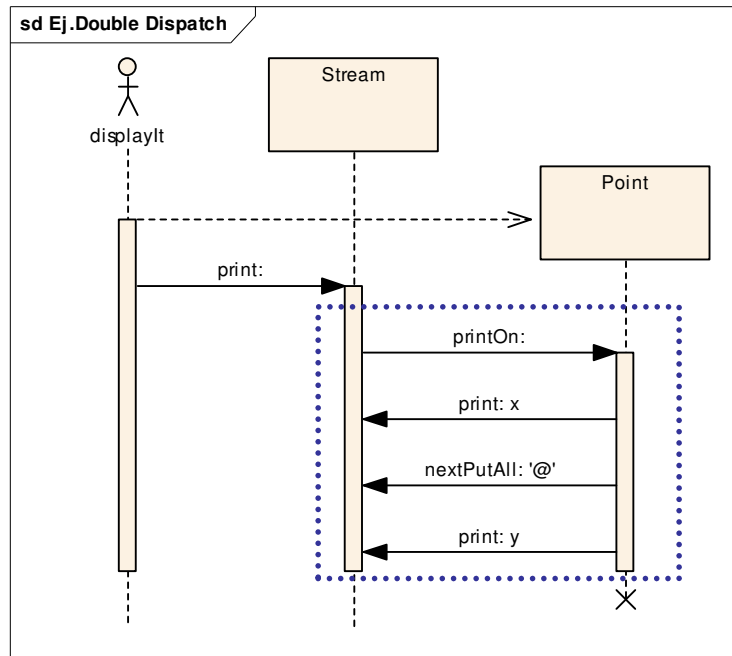
```
Stream>>print: anObject  
anObject printOn: self
```

```
Point>>printOn: aStream  
aStream print: x; nextPutAll: '@'; print: y
```

Acá vemos que para mostrar un objeto participan:

- un objeto Stream que sabe cómo imprimir y
- cada objeto sabe qué es lo que quiere mostrar de sí mismo.

Ejemplo: en un Workspace pedimos evaluar el siguiente mensaje



Multiple dispatch

Qué pasaría si yo pudiera definir el método `validarPostulacion` de la siguiente manera:

```

public void validarPostulacion(EmpleadoDePlanta postulante) {
    ...
}

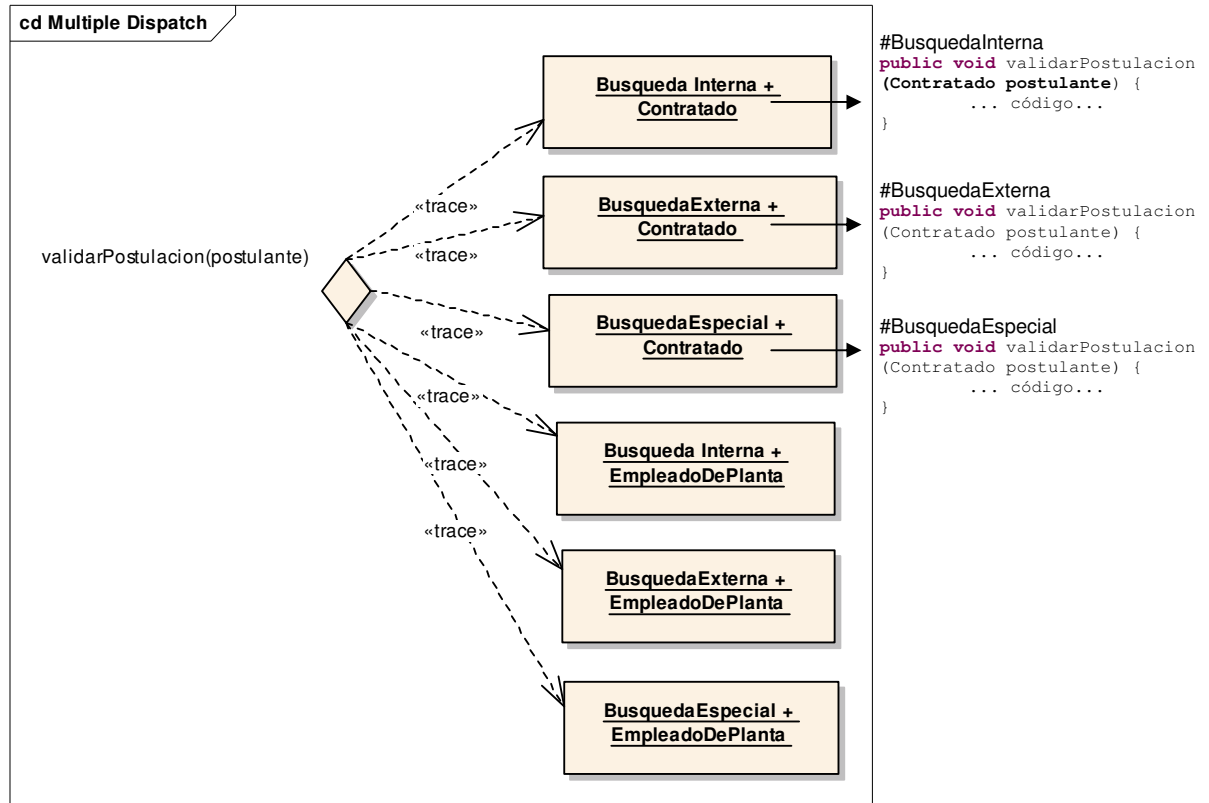
public void validarPostulacion(Contratado postulante) {
    ...
}

public void validarPostulacion(Externo postulante) {
    ...
}
  
```

y la VM decidiera en runtime qué método evaluar, en base a:

- la clase del objeto receptor **y**
- en base a la clase *posta* de los parámetros.

O sea, incluir en el mecanismo de `method lookup` a los parámetros del mensaje.



O visto matricialmente:

	Empleado de planta	Contratado	Externo
Búsqueda Interna	Validación 1	Validación 2	Validación 3
Búsqueda Externa	Validación 4	Validación 5	Validación 6
Búsqueda Especial	Validación 7	Validación 8	Validación 9

Claro, esto si tengo en cuenta **no el tipo**, sino la clase concreta que estoy recibiendo, se llama multimethods y lo pueden ver en <http://c2.com/cgi/wiki?MultiMethods>

Ojo, no estamos hablando de sobrecarga, esto es tener en cuenta el objeto en Runtime.

<u>Sobrecarga de métodos</u>	<u>Multimétodos</u>
Un método admite varias definiciones con distinta cantidad de parámetros o bien parámetros de distinto tipo. El binding es estático, en tiempo de compilación puedo saber exactamente qué método se va a ejecutar, no hay polimorfismo.	Un método admite varias definiciones con parámetros de diferentes clases. En tiempo de ejecución la VM determina qué método ejecutar en base al objeto receptor y a las clases concretas de los parámetros. Hay polimorfismo.

Algunos lenguajes que soportan multimétodos: Common Lisp, Dylan, Nice, Scheme, Slate.